

Space-Time Forecasting of Dynamic Scenes with Motion-aware Gaussian Grouping

Junmyeong Lee^{1*} Hoseung Choi^{1*} Minsu Cho^{1,2}

{junmyeong.lee, hs.choi, mscho}@postech.ac.kr

¹ Pohang University of Science and Technology (POSTECH) ² RLWRLD

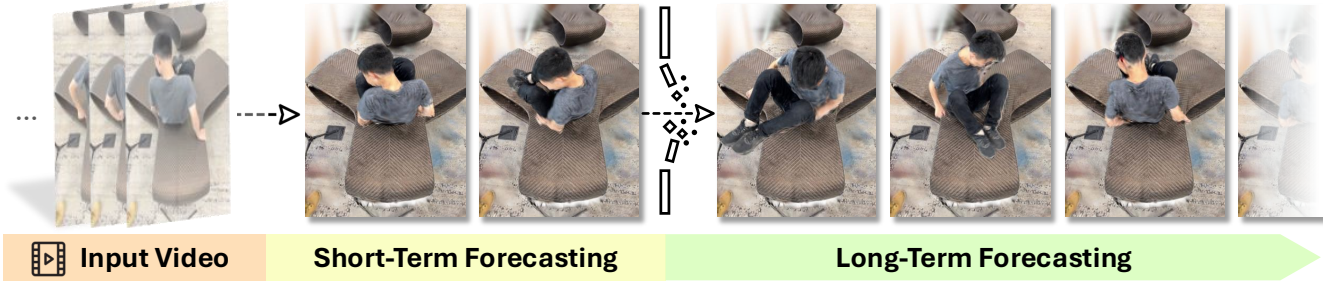


Figure 1. **Motion Group-aware Gaussian Forecasting (MoGaF)**. Our method predicts future frames of a dynamic input video by constructing object-level groups of Gaussian splatting with distinct motion patterns and leveraging these motion group structures in both space and time. The proposed method enables long-term, high-fidelity predictions on real-world videos with complex dynamics.

Abstract

Forecasting dynamic scenes remains a fundamental challenge in computer vision, as limited observations make it difficult to capture coherent object-level motion and long-term temporal evolution. We present **Motion Group-aware Gaussian Forecasting (MoGaF)**, a framework for long-term scene extrapolation built upon the 4D Gaussian Splatting representation. MoGaF introduces motion-aware Gaussian grouping and group-wise optimization to enforce physically consistent motion across both rigid and non-rigid regions, yielding spatially coherent dynamic representations. Leveraging this structured space-time representation, a lightweight forecasting module predicts future motion, enabling realistic and temporally stable scene evolution. Experiments on synthetic and real-world datasets demonstrate that MoGaF consistently outperforms existing baselines in rendering quality, motion plausibility, and long-term forecasting stability. Our project page is available at <https://slime0519.github.io/mogaf>

1. Introduction

Reconstructing dynamic environments from limited observations is a fundamental challenge in computer vision. Recent advances such as 3D Gaussian Splatting [15] have en-

abled dynamic scene reconstruction from everyday videos and multi-view captures. However, most existing methods still focus on *interpolation* of observed dynamics, while *extrapolation* remains largely unexplored. The ability to extrapolate scene dynamics is increasingly important for emerging applications such as robotic behavior planning and autonomous driving, where anticipating unobserved future motion is critical for decision making. To bridge this gap, we aim to forecast future scene dynamics by leveraging optimized scene representation from available observations.

Earlier approaches to dynamic scene extrapolation [11, 19, 23, 41, 42] formulate it as video prediction, generating future frames from past observations. Robotic planning models [1, 13, 40] adopt a similar idea, using generative models to predict future visual states from current inputs. However, these 2D-based methods are restricted to fixed-view generation and often produce geometrically inconsistent results in complex scenes [28]. In contrast, 3D reconstruction-based methods [15, 27] model time-dependent motion within explicit 3D scene representation. However, these approaches remain fundamentally *interpolative*, offering little *extrapolative* capability beyond the observed timestamps. As a result, their predicted dynamics often degrade when extended outside the training range. [50] incorporates explicit motion modeling to improve short-term prediction but still struggles with long-term forecasting.

Forecasting long-term dynamics remains challenging

*Equal contribution.

due to limitations in both representation and architecture. At the representation level, prior methods [20, 37] shows that Gaussian primitives often move independently, causing spatially incoherent motion that accumulates over time. At the architectural level, existing extrapolation model [50] are typically short-term predictors that produce frozen or collapsing trajectories during long rollouts.

To address these issues, we propose **Motion Group-aware Gaussian Forecasting (MoGaF)**, a unified framework for physically consistent long-term extrapolation based on 4D Gaussian Splatting (4DGS). MoGaF introduces (1) motion-aware Gaussian grouping that identifies coherent motion groups, (2) group-wise optimization that enforces consistent motion for rigid and non-rigid regions, and (3) lightweight motion forecasting to extrapolate future dynamics from the 4DGS representation. Integrating these components into a single pipeline yields spatially coherent and temporally stable scene evolution, enabling realistic long-term forecasting that surpasses baselines in both visual quality and motion plausibility.

Our contributions are summarized as follows:

- We present **MoGaF**, a framework that integrates object-level motion modeling into dynamic Gaussian Splatting for long-term scene extrapolation.
- Motion-aware Gaussian grouping, group-wise optimization, and a lightweight forecaster that jointly yield physically consistent and temporally coherent scene evolution.
- Extensive experiments on synthetic and real-world datasets demonstrate that MoGaF surpasses existing baselines in the visual quality of future frame synthesis.

2. Related work

2.1. Dynamic Scene Reconstruction

Dynamic scene reconstruction aims to recover time-varying 3D structures and appearance from observations. Early approaches represent deformable geometry with explicit primitives such as meshes, point clouds, or parametric models. Inspired by the success of NeRF [27], neural field-based methods [29–31] extend implicit representation to model temporal deformation fields. Another line of work [2, 9, 21] encode dynamics implicitly within feature spaces.

3D Gaussian Splatting (3DGS) [15] offers an explicit representation, enabling real-time rendering through volumetric Gaussian primitives and an efficient rasterization pipeline. Building on 3DGS, several methods learn per-Gaussian trajectories or factorized motion bases to obtain 4D representation [18, 24, 39, 46, 47]. However, many of these approaches assume limited object and camera motion, often leading to reconstruction failures and artifacts in highly dynamic scenes. Recent efforts [20, 25, 37] mitigate these issues by introducing improved initialization and optimization strategies that leverage data-driven priors such

as dynamic masks [44], dense point tracking [7, 14], and depth estimation [45]. Building on this direction, MoGaF groups Gaussians into coherent motion units and enforces group-wise motion consistency during optimization.

2.2. 3D Gaussian Segmentation

With the emergence of reliable 2D foundation models for segmentation [3, 17, 32], recent work has explored extending their prior knowledge to obtain 3D-consistent segmentation. Some methods [4, 16, 51] distill 2D foundation-model features into neural-field representation. Another stream of research lifts 2D mask predictions into 3D for multi-view segmentation without retraining. However, models such as SAM [32] do not preserve instance identities across views. This limitation has led to approaches [3, 8, 48] that associate identities via cross-view cues such as user interaction or video object tracking [5]. Gaga [26] further proposes a 3D-aware mask association pipeline using a Gaussian memory bank and IoU-based merging. Although effective for static scenes, these methods remain difficult to extend to dynamic settings due to temporal object motion.

In this work, we propose a segmentation method for 4DGS representation by extending the 3DGS-based segmentation pipeline [26] to dynamic scenes. This extension yields spatiotemporally coherent Gaussian groups. Our grouping strategy relies on alternating spatiotemporal region growing and keyframe-based Gaussian registration. This procedure is simple yet effective, enabling reliable grouping on an optimized 4DGS representation.

2.3. Dynamic Scene Forecasting

To extrapolate scene dynamics, several prior studies [22, 38] focused on pixel-space video prediction using convolutional or recurrent architectures, where future frames are generated directly in the image domain. Following this direction, generative models [43, 49] leverage transformer-based architectures for scalable video forecasting, and subsequent work [12, 35] further incorporates masked video modeling and self-supervised pretraining.

Building on recent progress in 3D reconstruction, another line of work predicts future dynamics directly in 3D space. This direction has recently been explored in the 4DGS setting as well, including GaussianPrediction (GSPred) [50], which forecasts keypoint motion using a graph-based network and propagates it to all Gaussians, and a concurrent work [36] that models continuous Gaussian motion via neural differential equations. Despite these advances, existing 3D-based models remain effective only for short-term prediction and often degrade significantly when extended to longer horizons. To address this limitation, our method performs object-wise forecasting and preserves both rigid and non-rigid geometry, enabling structurally consistent and high-fidelity rendering.

3. Preliminary

Gaussian Splatting. 3DGS [15] represents scene geometry using a set of Gaussian primitives and achieves real-time, high-fidelity rendering through efficient tile-based rasterization. Following [37], each Gaussian $g \in \mathcal{G}$ is parameterized in a canonical space by $\{\boldsymbol{\mu}, \mathbf{R}, s, o, c\}$, which specify its mean, rotation, scale, opacity, and color. Here, $\boldsymbol{\mu} \in \mathbb{R}^3$ and $\mathbf{R} \in SO(3)$ denote the 3D mean and rotation, while $s \in \mathbb{R}^3$, $o \in \mathbb{R}$, and $c \in \mathbb{R}^3$ represent its scale, opacity, and color.

According to [52], Gaussians are projected onto the image plane by approximating their 2D means and covariances as follows:

$$\boldsymbol{\mu}^{2D} = \Pi(\mathbf{K}\mathbf{E}\boldsymbol{\mu}), \quad \boldsymbol{\Sigma}^{2D} = \mathbf{J}\mathbf{E}\boldsymbol{\Sigma}\mathbf{E}^\top \mathbf{J}^\top, \quad (1)$$

where $\boldsymbol{\Sigma}$ and $\boldsymbol{\Sigma}^{2D}$ denote the 3D and 2D covariance matrices, respectively. $\mathbf{J}$ is the Jacobian of the affine approximation of the projective transformation, and $\mathbf{K}$ and $\mathbf{E}$ are the intrinsic and extrinsic matrices of the camera. Π denotes the perspective projection from 3D points to the image plane. Each covariance matrix is decomposed into a rotation matrix $\mathbf{R}$ and a scaling matrix $\mathbf{S}$ such that

$$\boldsymbol{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^\top \mathbf{R}^\top.$$

In addition, each Gaussian includes an opacity $\alpha \in \mathbb{R}$ and spherical harmonics (SH) coefficients $c \in \mathbb{R}^{(L+1)^2}$ to represent view-dependent color. Thus, the final color of pixel x_p is computed as:

$$C_p = \sum_{i=1}^N c_i \alpha_i T_i \mathcal{N}(x_p | \boldsymbol{\mu}^{2D}, \boldsymbol{\Sigma}^{2D}), \quad (2)$$

where $T_i = \prod_{j=1}^{i-1} (1 - \alpha_j \mathcal{N}(x_p | \boldsymbol{\mu}^{2D}, \boldsymbol{\Sigma}^{2D}))$, and c_i and α_i represent color and opacity associated with each 3D Gaussian, respectively.

Extension for Dynamic Scene. Then, the time-dependent motion parameters $\boldsymbol{\mu}_t$ and $\mathbf{R}_t$ are obtained by composing the canonical Gaussian parameters—mean $\boldsymbol{\mu}_c$, rotation $\mathbf{R}_c$ —with the motion transformation $\mathbf{T}_{c \rightarrow t} = [\mathbf{R}_{c \rightarrow t} \mid \mathbf{t}_{c \rightarrow t}] \in SE(3)$, defined as:

$$\boldsymbol{\mu}_t = \mathbf{R}_{c \rightarrow t} \boldsymbol{\mu}_c + \mathbf{t}_{c \rightarrow t}, \quad \mathbf{R}_t = \mathbf{R}_{c \rightarrow t} \mathbf{R}_c. \quad (3)$$

For each timestep t , we represent Gaussian motion using a set of B motion bases $\{\mathbf{T}_{c \rightarrow t}^{(b)}\}_{b=1}^B$ shared across all timesteps $t = 0, 1, \dots, T$. Each Gaussian is assigned a blend weight (i.e., motion coefficient) $w \in \mathbb{R}^B$ to combine these bases and represent its motion. At timestep t , the composed motion transformation is computed as:

$$\mathbf{T}_{c \rightarrow t} = \sum_{b=1}^B w^{(b)} \mathbf{T}_{c \rightarrow t}^{(b)}, \quad (4)$$

where $\|w\| = 1$. During optimization, both the motion bases $\{\mathbf{T}_{c \rightarrow t}^{(b)}\}_{b=1}^B$ and the motion coefficient $w^{(b)}$ are jointly optimized with the other Gaussian properties. The remaining rendering process follows the 3DGS formulation, using the deformed rotation $\mathbf{R}_t$ and mean vector $\boldsymbol{\mu}_t$.

4. Methods

Our method, MoGaF, takes as input a casually captured dynamic video $\{I_t \in \mathbb{R}^{H \times W \times 3}\}_{t=1}^T$ and forecasts scene dynamics by rendering novel frames for unseen timesteps ($t > T$). Unlike previous methods [36, 50], our goal is to achieve realistic and physically consistent long-term forecasting at the scene level.

We address the limitations of dynamic 3D representation by reconstructing scene dynamics through a constrained optimization process that enforces physically consistent motion across time. MoGaF models object-level motion explicitly within the dynamic Gaussian splatting framework.

The overall pipeline consists of three stages: (i) motion-aware Gaussian grouping, which identifies coherent motion groups and labels each group as either rigid or non-rigid (Section 4.1); (ii) group-wise constrained optimization, which applies type-specific motion regularization to produce a physically structured 4D representation with object-level consistency (Section 4.2); and (iii) group-wise long-term motion forecasting, where a temporal prediction module extrapolates future motion trajectories and renders temporally coherent scene evolution (Section 4.3). An overview of our framework is illustrated in Figure 2.

4.1. Motion-aware Gaussian Grouping

Given the frames $\{I_t\}_{t=1}^T$ and the reconstructed 4DGS representation, we group Gaussians into distinct groups with coherent motion and label each group as *rigid* or *non-rigid* to support subsequent motion-constrained optimization and forecasting. Inspired by [26], we design a memory-bank mechanism adapted to the optimized 4DGS representation. Unlike prior 3DGS-based grouping methods, our module handles dynamic representation and explicitly distinguishes motion types of groups. Given K object groups, we represent them as $\mathcal{M}^{(k)} = (G^{(k)}, \tau^{(k)})$, where $G^{(k)}$ is the set of Gaussians in the k -th group and $\tau^{(k)} \in \{0, 1\}$ indicates its motion type (0 for *non-rigid* and 1 for *rigid*). An overview of the grouping algorithm is shown in Algorithm 1. Detailed analyses of our Gaussian grouping are provided in the Supplementary Material (Sections A and D.2).

Leverage 2D Segmentation Prior. Similar to previous methods [26, 48], we first obtain 2D masks for the input video. In particular, we use a grounded segmentation model [33] to simultaneously produce K object masks and their corresponding rigidity labels $\tau^{(k)}$. Following [26], we then identify Gaussians that contribute to rendering each

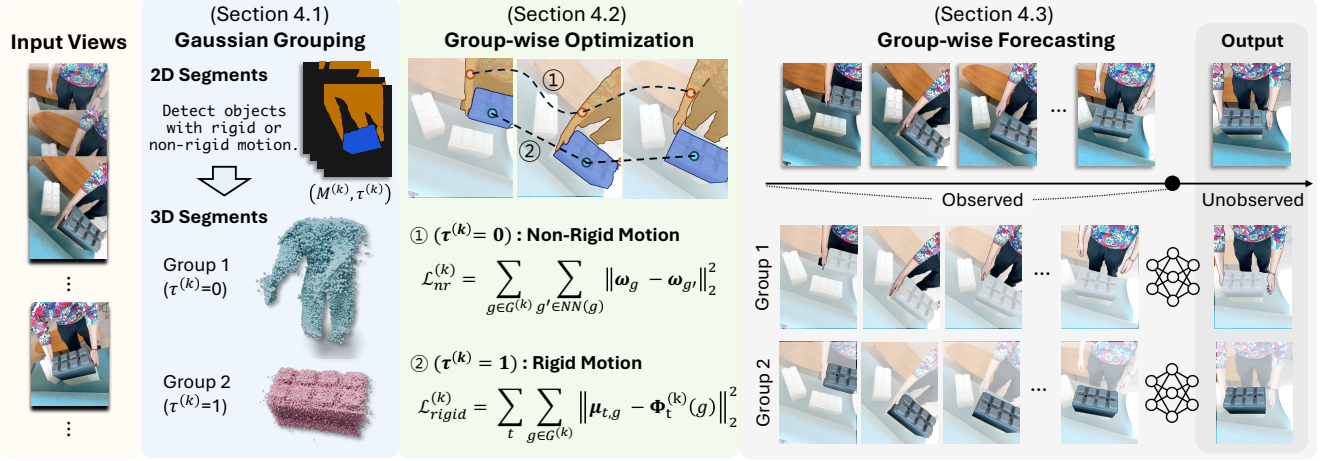


Figure 2. **Overall pipeline of MoGaF.** Given a video, MoGaF generates future frames of the scene. To achieve realistic forecasting, our method builds on 4DGS representation and proceeds as follows: (1) **Gaussian Grouping**: Gaussians are clustered into motion-consistent object groups, with each group labeled as rigid or non-rigid using grounded 2D segmentation. (2) **Group-wise Optimization**: Grouped Gaussians are refined with rigidity-aware motion constraints: rigid groups are guided by a shared $SE(3)$ transform, while non-rigid groups are regularized with local motion smoothness. (3) **Group-wise Forecasting**: For each group, a lightweight Transformer-based forecaster extrapolates Gaussian trajectories beyond the observed frames, enabling rendering at novel viewpoints for future timesteps.

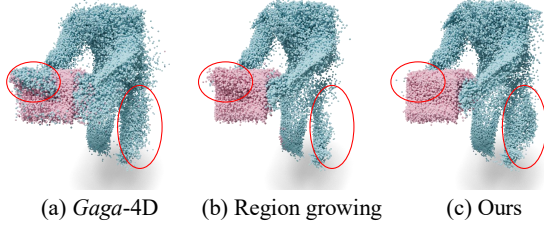


Figure 3. **Result of Gaussian grouping.** Compared to (a) simple extension of 3DGS grouping [26] and (b) single-frame mask-based region growing, our hybrid approach produces complete and reliable motion-aware Gaussian groups.

masked region and extract these front Gaussians along the viewing direction.

Simple Extension of Static Gaussian Grouping. A straightforward way to extend static Gaussian grouping [26] to dynamic scenes is to associate each 2D mask $M_t^{(k)}$ with the set of deformed Gaussians whose projected positions fall inside the mask:

$$G_t^{(k)} = \{g \in \mathcal{G} \mid \text{Proj}(g_t) \in M_t^{(k)}\}, \quad (5)$$

where g_t denotes the deformed Gaussian state at timestep t . We then update each Gaussian group $G^{(k)} \in \mathcal{M}^{(k)}$ in the memory bank by merging $G_t^{(k)}$ when sufficient spatial overlap is observed. However, because this association relies solely on deformed spatial positions of Gaussians, it often leads to misgrouping (Figure 3a), especially under occlusion or when Gaussians from different objects overlap.

Gaussian Grouping via Iterative Region Growing. To overcome the limitations of this simple algorithm, we adopt an iterative region-growing strategy that alternates between

keyframe-based seeding and feature-space expansion. At each keyframe, we project the deformed Gaussians and select the front-most ones that contribute to rendering each mask as reliable seeds $G_t^{(k)}$. These seeds are merged into their corresponding motion groups $G^{(k)}$ in the memory bank, which then serve as the starting points for region growing. During region growing, each Gaussian $g \in \mathcal{G}$ is represented by a compact spatiotemporal feature $\mathbf{f}_g = [\mu_{c,g}, \mathbf{w}'_g]$, where $\mu_{c,g}$ denotes its canonical-space mean and $\mathbf{w}'_g$ encodes its PCA-reduced motion coefficient. Using these features, each group $G^{(k)}$ is expanded by aggregating nearby Gaussians that satisfy $|\mathbf{f}_g - \mathbf{f}_{g'}| < \epsilon_r$. The updated groups are stored in the memory bank, and the process continues at the next keyframe.

Through this alternating loop of front-Gaussian seeding and feature-space region growing, our method effectively captures the spatiotemporal characteristics of the dynamic Gaussian representation, resulting in robust and accurate motion grouping. As shown in Figure 3, our approach produces reliable object-level motion groups with highly accurate assignments, outperforming competing methods.

4.2. Group-wise Gaussian Motion Optimization

After grouping, we further optimize the 4DGS representation by incorporating the object group information. We introduce group-wise motion constraints guided by the rigidity flag $\tau^{(k)}$, which adaptively applies distinct motion regularization strategies to rigid and non-rigid object groups.

Rigid Motion Anchoring Loss. We perform constrained optimization to refine the grouped 4DGS representation by adaptively applying motion constraints to each Gaussian $g \in G^{(k)}$ according to the group motion type. For

Algorithm 1: Motion-aware Gaussian Grouping

Input: Video frames $\{I_t\}_{t=1}^T$, segmentation model $\mathcal{S}$, dynamic Gaussian set $\mathcal{G}$ with deformed Gaussian states $\{g_t\}_{t=1}^T$
Output: Motion groups $\{\mathcal{M}^{(k)} = (G^{(k)}, \tau^{(k)})\}_{k=1}^K$

Stage 1: Init at the First Keyframe

Obtain $\{M_t^{(k)}, \tau_{\text{mask}}^{(k)}\} \leftarrow \mathcal{S}(\{I_t\})$

Select first keyframe t_k ,

foreach $k = 1, \dots, K$ **do**

$G^{(k)} \leftarrow \{g \in \mathcal{G} \mid \text{Proj}(g_{t_k}) \in M_{t_k}^{(k)}\}$
 $\tau^{(k)} \leftarrow \tau_{\text{mask}}^{(k)}$

Define features for all g : $\mathbf{f}_g = [\mu_{c,g}, \mathbf{w}'_g]$

Stage 2: Iterative Region Growing

foreach keyframe t in stride order **do**

 // (A) Feature-space region growing
 repeat
 $U \leftarrow \mathcal{G} \setminus (\bigcup_k G^{(k)})$
 foreach $k = 1, \dots, K$ **do**
 $\epsilon_k \leftarrow \alpha \cdot \text{mean}_{g \in G^{(k)}} \text{KNN}_K(\mathbf{f}_g)$
 $N_k \leftarrow \{g' \in U \mid \min_{g \in G^{(k)}} \|\mathbf{f}_g - \mathbf{f}_{g'}\| < \epsilon_k\}$
 $G^{(k)} \leftarrow G^{(k)} \cup N_k$
 $U \leftarrow U \setminus N_k$
 until no group changes;

 // (B) Seeding & merge at keyframe t

foreach $k = 1, \dots, K$ **do**

$G_t^{(k)} \leftarrow \{g \mid \text{Proj}(g_t) \in M_t^{(k)}\}$
 $G^{(k)} \leftarrow G^{(k)} \cup G_t^{(k)}$

a group labeled as rigid ($\tau^{(k)} = 1$), all Gaussians within the group are constrained to share a single $SE(3)$ transformation $\Phi_t^{(k)} = [\mathbf{R}_{c \rightarrow t}^{(k)} \mid \mathbf{t}_{c \rightarrow t}^{(k)}]$. We define the group-level rigid transformation as a function acting on a Gaussian $g \in G_k$ that maps its canonical-space mean to the deformed space at time t :

$$\Phi_t^{(k)}(g) = \mathbf{R}_{c \rightarrow t}^{(k)} \mu_{c,g} + \mathbf{t}_{c \rightarrow t}^{(k)}. \quad (6)$$

This operator ensures that all Gaussians within the same rigid group undergo a consistent $SE(3)$ motion, preserving intra-group spatial structure during optimization. The rigid-body loss then measures the deviation between the learned per-Gaussian motion and the group-level rigid transform:

$$\mathcal{L}_{\text{rigid}}^{(k)} = \sum_t \sum_{g \in G^{(k)}} \|\mu_{t,g} - \Phi_t^{(k)}(g)\|_2^2. \quad (7)$$

Non-Rigid Motion Smoothness Loss. For non-rigid groups ($\tau^{(k)} = 0$), each Gaussian $g \in \mathcal{G}$ is assigned a learnable motion coefficient $\mathbf{w}_g \in \mathbb{R}^B$ that governs its deformation behavior over time. Following [37], we promote locally

coherent motion in the canonical space by applying a spatial smoothness regularization defined as

$$\mathcal{L}_{\text{nr}}^{(k)} = \sum_{g \in G^{(k)}} \sum_{g' \in \text{NN}(g)} \|\mathbf{w}_g - \mathbf{w}_{g'}\|_2^2, \quad (8)$$

where $\text{NN}(g)$ denotes the spatial neighbors of Gaussian g , typically selected within a fixed-radius region in the canonical space. This regularization encourages spatially adjacent Gaussians to share consistent motion coefficients and preserves locally smooth motion patterns across the canonical space. Finally, the overall motion objective integrates both terms under the rigidity flag:

$$\mathcal{L}_{\text{motion}} = \sum_{k=1}^K \left[\tau^{(k)} \mathcal{L}_{\text{rigid}}^{(k)} + (1 - \tau^{(k)}) \mathcal{L}_{\text{nr}}^{(k)} \right]. \quad (9)$$

Additional optimization objectives are described in the supplementary material. With the object motion-aware optimization stage, we obtain a refined 4D Gaussian representation where rigid groups follow coherent $SE(3)$ motion and non-rigid groups exhibit locally smooth deformation. This reduces per-Gaussian drift and improves temporal consistency of the reconstructed motion.

4.3. Group-wise Motion Forecasting

After obtaining motion-consistent 4DGS representation, we aim to forecast the future evolution of Gaussian motion beyond the observed frames. At time t , the motion of a Gaussian $g \in \mathcal{G}$ is represented by its $SE(3)$ transformation $\mathbf{T}_{t,g} = [\mathbf{R}_{t,g} \mid \mu_{t,g}]$. Similar to [36, 50], our forecaster takes the optimized motion sequence from 4DGS representation, $\{\mathbf{T}_{t,g}\}_{t=0}^T$, as input and predicts its future continuation $\{\hat{\mathbf{T}}_{t,g}\}_{t>T}$. The forecasting is performed in an autoregressive rollout, iteratively using the latest $T-1$ frames including predictions to generate subsequent timesteps.

Masked Input Training. The motion forecaster adopts a shallow Transformer encoder [34], which effectively models temporal dependencies but tends to overfit when trained on fully observed sequences. To enhance generalization and temporal reasoning, we introduce a masked motion modeling strategy inspired by masked language modeling in NLP [6]. This segment-level masking encourages the model to infer missing temporal dynamics from surrounding motion context, resulting in better temporal understanding. It further improves long-horizon prediction and robustness to noisy inputs. The masking ratio is gradually annealed during training to match the inference condition.

Group-wise Forecasting. We train a lightweight motion forecaster to each motion group $G^{(k)}$ obtained from the group-wise optimization in Section 4.2, enabling the model to learn group-specific temporal dynamics. This group-wise formulation enhances both stability and accuracy of motion

Table 1. **Forecasting results on the iPhone dataset.** We forecast frames beyond the observed time window and evaluate the predicted frames rendered from held-out test viewpoints. The leftmost column (*Obs. ratio*) indicates the fraction of input training frames used by each model; 80% and 60% correspond to forecasting the remaining 20% and 40% of frames, respectively. [†] denotes methods that are re-implemented for the SoM-based 4DGS representation and retain their original forecasting scheme.

Obs. ratio	Scene	GSPred [50]			GSPred-SoM [†] [50]			ODE-GS-SoM [†] [36]			MoGaF (Ours)		
		mPSNR [↑]	mSSIM [↑]	mLPIPS [↓]	mPSNR [↑]	mSSIM [↑]	mLPIPS [↓]	mPSNR [↑]	mSSIM [↑]	mLPIPS [↓]	mPSNR [↑]	mSSIM [↑]	mLPIPS [↓]
80%	apple	15.75	0.5589	0.4338	17.22	0.8122	0.4806	16.35	0.8135	0.4803	17.97	0.8199	0.4341
	block	12.61	0.5220	0.5363	12.20	0.5807	0.5352	12.07	0.5717	0.5583	12.62	0.5875	0.5352
	paper-windmill	14.76	0.2244	0.4329	19.16	0.5573	0.2142	19.13	0.5512	0.2154	19.34	0.5552	0.2052
	spin	14.54	0.4652	0.4002	15.12	0.6701	0.3166	15.69	0.6593	0.3354	15.96	0.6740	0.3236
	teddy	11.15	0.5790	0.5752	11.26	0.5823	0.6942	10.06	0.5821	0.7093	11.99	0.5607	0.6157
	Average	13.76	0.4699	0.4757	14.99	0.6405	0.4482	14.66	0.6355	0.4597	15.58	0.6395	0.4227
60%	apple	14.44	0.5682	0.6108	16.96	0.7460	0.4491	15.48	0.7333	0.5035	16.57	0.7409	0.4660
	block	13.61	0.5494	0.5242	13.02	0.6022	0.5231	13.94	0.6101	0.5314	14.06	0.6134	0.5055
	paper-windmill	14.96	0.2254	0.4651	18.30	0.4888	0.2345	17.94	0.4745	0.2566	18.34	0.4858	0.2308
	spin	14.52	0.4564	0.4203	15.79	0.6719	0.3192	14.89	0.6490	0.3508	16.47	0.6838	0.3159
	teddy	11.49	0.5828	0.5753	11.63	0.5447	0.6020	11.96	0.5526	0.6010	12.12	0.5478	0.6045
	Average	13.80	0.4764	0.5192	15.14	0.6107	0.4256	14.84	0.6039	0.4487	15.51	0.6143	0.4245

forecasting by decoupling heterogeneous motion dynamics across objects. Within each group, the forecaster captures consistent temporal patterns shared among Gaussians exhibiting similar motion behavior, leading to smoother and more coherent predictions.

Training Objectives. For each motion group $G^{(k)}$, the forecaster takes the Gaussian motion up to time $T-1$ as input and is trained to minimize the discrepancy between the predicted and observed motion at time T . The motion reconstruction loss is defined as

$$\mathcal{L}_{\text{pred}}^{(k)} = \frac{1}{|G^{(k)}|} \sum_{g \in G^{(k)}} \|\mathbf{T}_{T,g} - \hat{\mathbf{T}}_{T,g}\|_2^2. \quad (10)$$

To enforce physically smooth motion, we apply an acceleration regularization term:

$$\mathcal{L}_{\text{acc}}^{(k)} = \frac{1}{|G^{(k)}|} \sum_{g \in G^{(k)}} \|\hat{\boldsymbol{\mu}}_{T,g} - 2\boldsymbol{\mu}_{T-1,g} + \boldsymbol{\mu}_{T-2,g}\|_2^2, \quad (11)$$

where $\hat{\boldsymbol{\mu}}_{t,g}$ denotes the predicted mean of Gaussian g at time t . The overall training objective for $G^{(k)}$ is given by:

$$\mathcal{L}_{\text{group}}^{(k)} = \mathcal{L}_{\text{pred}}^{(k)} + \lambda_{\text{acc}} \mathcal{L}_{\text{acc}}^{(k)}, \quad (12)$$

and each forecaster is trained independently using $\mathcal{L}_{\text{group}}^{(k)}$.

5. Experiments

5.1. Implementation Details

We build our dynamic reconstruction pipeline upon Shape-of-Motion (SoM) [37], which serves as the backbone for canonical space and motion parameterization. For Gaussian grouping, we extend the official implementation of Gaga [26] and integrate it into our framework. For motion forecasting, we design a lightweight transformer-based forecaster consisting of a single-layer encoder with eight attention heads, a 32-dimensional feature embedding, and a

64-dimensional feed-forward expansion. A final prediction head maps the encoded features to future motion parameters. Additional architectural and optimization details are provided in the Supplementary Material (Section B).

5.2. Future View Synthesis

We evaluate future view synthesis performance following the protocol of GSPred [50]. Specifically, we use a subset of the original training frames as input observations and measure forecasting performance on the remaining unseen frames. We consider two observation ratios: 80% (as in GSPred) and a more challenging 60% setting to assess long-term prediction capability.

Evaluation on iPhone dataset. We first evaluate our method on iPhone dataset [10], which contains hundreds of frames of dynamic scenes captured from multiple hand-held cameras. Since GSPred [50] is built on [39], it does not leverage data-driven priors such as depth estimation [45] or dense point tracking [7, 14], and thus often underperforms on highly dynamic monocular captures. For a fair comparison, we adopt the GCN-based forecasting architecture from GSPred and train it on top of SoM, denoted as *GSPred-SoM*. Additionally, following [36], we reproduce a latent neural ODE forecaster on the SoM representation, denoted as *ODE-GS-SoM*.

As shown in Table 1, MoGaF consistently achieves higher photometric fidelity than the baselines. Figure 4 shows that MoGaF preserves the geometry of both rigid and non-rigid dynamic objects, producing realistic future renderings by accurately extrapolating the observed motion. In contrast, baselines often exhibit geometric inconsistency and unrealistic dynamics in scenes involving large or rapid motion. To further assess long-term behavior, we examine the forecasting results for *apple* in Figure 6. GSPred renders the hand as nearly static in the future frames despite its highly dynamic motion in the observations, while GSPred-SoM and ODE-GS-SoM fail to maintain object geometry.

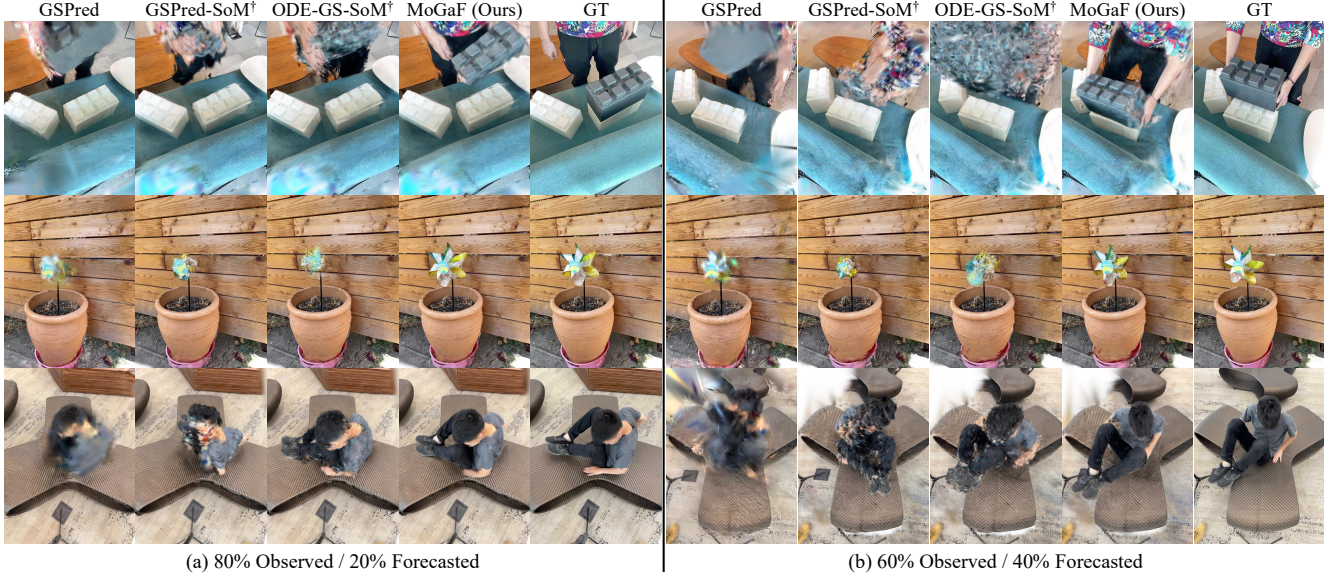


Figure 4. **Qualitative results on iPhone dataset.** We present forecasted frames from test camera views. (a) and (b) correspond to settings where the first 80% and 60% of frames are used for training, and the remaining 20% and 40% are forecasted, respectively.

Table 2. **Forecasting results on D-NeRF dataset.** We use 60% of the frames as training and predict the remaining 40%.

Scene	GSPred [50]			Ours		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Trex	20.63	0.9271	0.0627	20.80	0.9379	0.0551
Mutant	25.14	0.9375	0.0415	25.69	0.9246	0.0482
Jumpingjacks	19.84	0.9087	0.0924	19.86	0.9090	0.0920
Standup	21.99	0.9170	0.0796	22.32	0.9201	0.0756
Bouncingballs	23.68	0.9603	0.0540	25.03	0.9623	0.0436
Hook	21.18	0.8855	0.0851	22.41	0.8929	0.0784
Hellwarrior	29.11	0.9056	0.0957	29.23	0.9075	0.0894
Lego	12.65	0.7667	0.2244	21.61	0.8631	0.1142
Average	21.78	0.9011	0.0919	23.37	0.9147	0.0746

In comparison, MoGaF accurately preserves object geometry and predicts the hand’s complex motion, demonstrating its effectiveness in extrapolating coherent dynamics. Additional results are included in the Supplementary Material.

Evaluation on D-NeRF dataset. We also evaluate forecasting performance on D-NeRF dataset [31], which consists of synthetic frames capturing dynamic objects rendered from randomly sampled viewpoints on a hemisphere at each timestep. Since D-NeRF dataset consists solely of object-level scenes, we follow the same 4DGS training procedure as GSPred [50] to enable a fair comparison of the forecasting module in MoGaF. As shown in Table 2, MoGaF achieves higher performance across most scenes, demonstrating stronger temporal coherence and long-range motion modeling under the 60%–40% forecasting setting. Figure 5 further presents qualitative results under the 80%–20% split. In *Standup*, MoGaF captures the global motion trend and produces realistic long-term forecasts. In *Lego*, it preserves object structure and motion, whereas GSPred often loses shape consistency. Additional results are provided in the Supplementary Material.

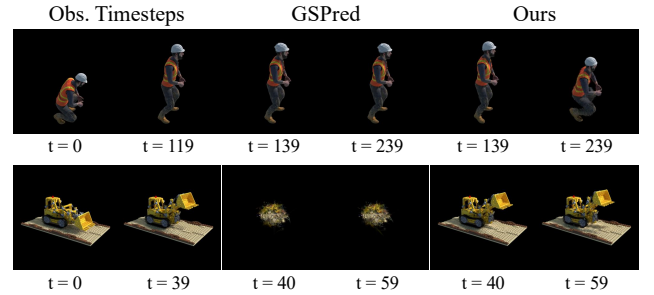


Figure 5. **Forecasting results on D-NeRF dataset.** We render extrapolated future frames. Note that in *Obs. Timesteps*, the first and second columns show renderings reconstructed from training views at the first and last observed timesteps, respectively.

5.3. Ablation Studies

Group-wise Optimization and Forecasting. We analyze the effectiveness of our group-wise optimization and forecasting. To this end, we remove the group-wise optimization stages and forecast the future motion of all dynamic Gaussians jointly with a single forecaster (i.e., without object-wise forecasters). We evaluate forecasting quality using 2D and 3D point-tracking metrics following the protocol of [37]. As shown in Table 3a, performance consistently degrades without the group-aware design. These results indicate that object-level structure is crucial for physically plausible, temporally coherent long-term forecasts. More detailed ablations of the group-wise modules are reported in the Supplementary Material D.1 (Table 7a).

Masked Motion Forecaster Training. We assess the effect of our motion-masking strategy on D-NeRF dataset by comparing the forecaster against a variant trained without masking. As shown in Table 3b and Figure 7, removing the masking strategy leads to noticeable performance degrada-

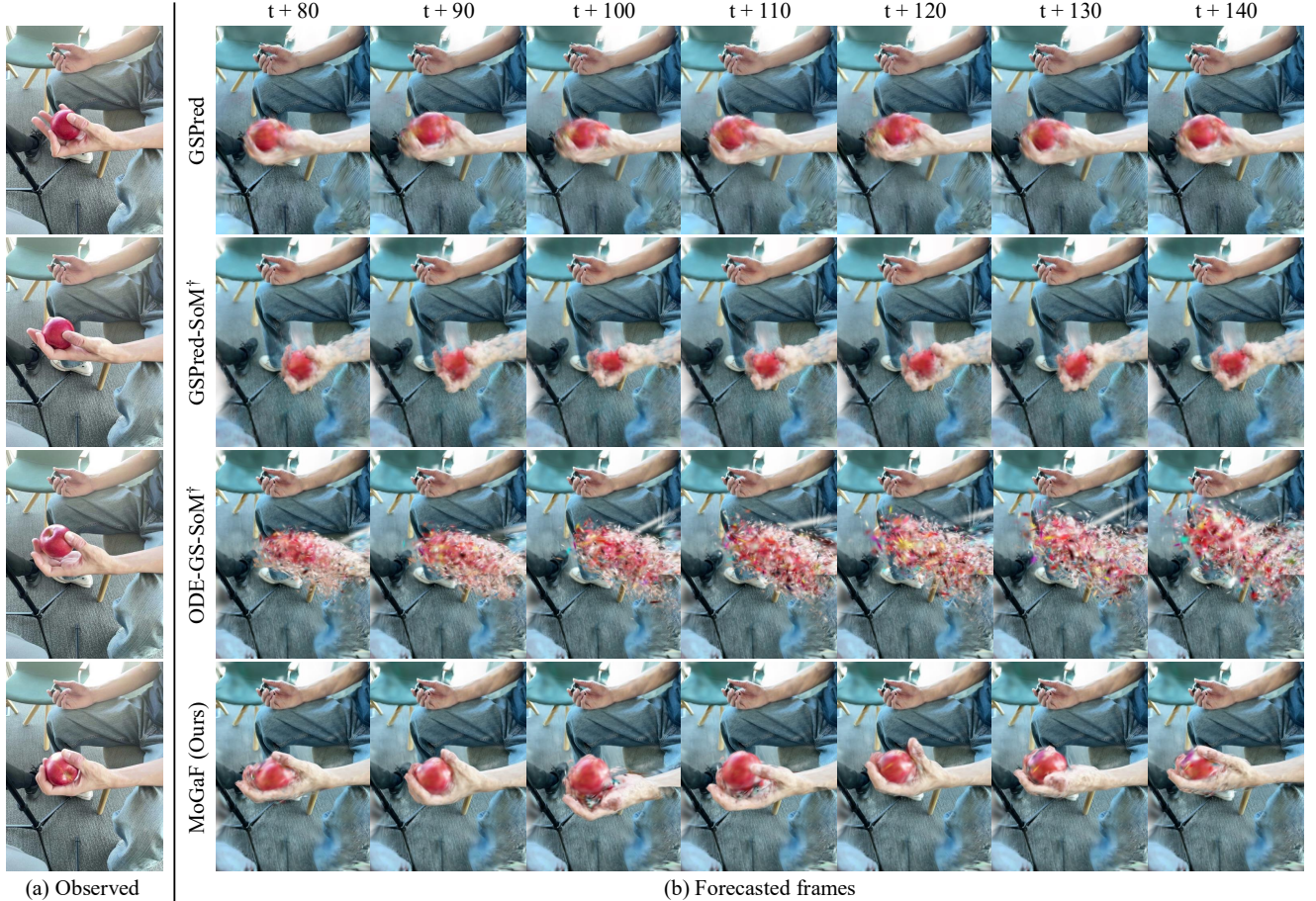


Figure 6. **Long-term forecasting results on iPhone dataset.** (a) shows GT views from test cameras at the observed timesteps, and (b) presents the forecasted renderings for timesteps beyond the observations. Note that t denotes the timestep of the last observed frame.

Table 3. **Ablation results.** (Top) Tracking performance on iPhone dataset. (Bottom) Masking performance on D-NeRF dataset.

(a) We evaluate point-tracking metrics on the forecasted 40% future frames with and without both group-wise optimization and forecasting.

Method	3D tracking			2D tracking		
	EPE↓	$\delta_{3D}^{10}\uparrow$	$\delta_{3D}^{05}\uparrow$	AJ↑	$\delta_{avg}\uparrow$	OA↑
Ours w/o grouping	0.296	35.6	17.1	13.3	8.5	64.1
MoGaF (Ours)	0.236	44.8	22.5	17.5	8.8	80.1

(b) We evaluate 20% future frame forecasting using forecasters trained with and without masking.

Method	PSNR↑	SSIM↑	LPIPS↓
Ours w/o masking	24.68	0.9283	0.0551
Ours	25.87	0.9357	0.0491

tion. This gap indicates that masking helps the encoder attend to intrinsic motion cues, resulting in more stable long-term predictions and improved robustness under extended forecasting horizons.

6. Conclusion

We introduced Motion Group-aware Gaussian Forecasting (MoGaF), a unified framework for dynamic scene long-

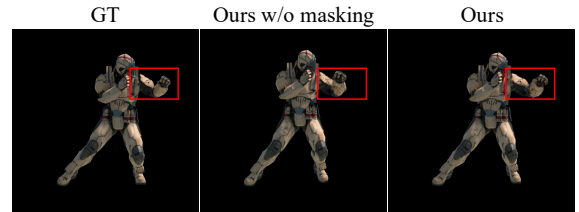


Figure 7. **Effect of masking.** Applying contiguous-span masking during training enhances the robustness of motion forecasting.

term extrapolation leveraging structured 4D representation. MoGaF explicitly models object-level dynamics through motion-aware Gaussian grouping, group-wise optimization, and a lightweight forecasting module. By enforcing physically consistent rigid and non-rigid motion within a structured space-time representation, MoGaF produces 4DGS models that are both spatially coherent and temporally stable, enabling realistic scene evolution beyond the observed frames. Extensive experiments on synthetic and real-world datasets show that MoGaF significantly improves long-term forecasting accuracy over existing baselines, and ablation studies confirm the contribution of each component to achieving spatial consistency and stable predictions.

Acknowledgement

This work was supported by the IITP grants (RS-2022-II220959: Few-Shot Learning of Causal Inference in Vision and Language for Decision Making (50%), RS-2022-II220264: Comprehensive Video Understanding and Generation (25%), RS-2022-II220290: Visual Intelligence for Space-Time Understanding and Generation based on Multi-layered Visual Common Sense (20%), RS-2019-II191906: AI Graduate School at POSTECH (5%)) funded by the Ministry of Science and ICT, Korea.

References

- [1] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1
- [2] Ang Cao and Justin Johnson. Hexplane: A fast representation for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 130–141, 2023. 2
- [3] Jiazhong Cen, Jiemin Fang, Zanwei Zhou, Chen Yang, Lingxi Xie, Xiaopeng Zhang, Wei Shen, and Qi Tian. Segment anything in 3d with radiance fields. *International Journal of Computer Vision*, pages 1–23, 2025. 2
- [4] Xiaokang Chen, Jiaxiang Tang, Diwen Wan, Jingbo Wang, and Gang Zeng. Interactive segment anything nerf with feature imitation. *arXiv preprint arXiv:2305.16233*, 2023. 2
- [5] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Alexander Schwing, and Joon-Young Lee. Tracking anything with decoupled video segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1316–1326, 2023. 2
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019. 5
- [7] Carl Doersch, Yi Yang, Mel Vecerik, Dilara Gokay, Ankush Gupta, Yusuf Aytar, Joao Carreira, and Andrew Zisserman. Tapir: Tracking any point with per-frame initialization and temporal refinement. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10061–10072, 2023. 2, 6
- [8] Bin Dou, Tianyu Zhang, Zhaohui Wang, Yongjia Ma, Zejian Yuan, and Nanning Zheng. Learning segmented 3d gaussians via efficient feature unprojection for zero-shot neural scene segmentation. In *International Conference on Neural Information Processing*, pages 398–412. Springer, 2024. 2
- [9] Sara Fridovich-Keil, Giacomo Meanti, Frederik Rahbæk Warburg, Benjamin Recht, and Angjoo Kanazawa. K-planes: Explicit radiance fields in space, time, and appearance. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12479–12488, 2023. 2
- [10] Hang Gao, Ruilong Li, Shubham Tulsiani, Bryan Russell, and Angjoo Kanazawa. Monocular dynamic view synthesis: A reality check. *Advances in Neural Information Processing Systems*, 35:33768–33780, 2022. 6, 2, 5, 7
- [11] Rohit Girdhar and Kristen Grauman. Anticipative video transformer. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 13505–13515, 2021. 1
- [12] Agrim Gupta, Stephen Tian, Yunzhi Zhang, Jiajun Wu, Roberto Martín-Martín, and Li Fei-Fei. Maskvit: Masked visual pre-training for video prediction. *arXiv preprint arXiv:2206.11894*, 2022. 2
- [13] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019. 1
- [14] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Co-tracker: It is better to track together. In *European conference on computer vision*, pages 18–35. Springer, 2024. 2, 6
- [15] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. 1, 2, 3
- [16] Chung Min Kim, Mingxuan Wu, Justin Kerr, Ken Goldberg, Matthew Tancik, and Angjoo Kanazawa. Garfield: Group anything with radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21530–21539, 2024. 2
- [17] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023. 2
- [18] Agelos Kratimenos, Jiahui Lei, and Kostas Daniilidis. Dynmf: Neural motion factorization for real-time dynamic view synthesis with 3d gaussian splatting. In *European Conference on Computer Vision*, pages 252–269. Springer, 2024. 2
- [19] Yong-Hoon Kwon and Min-Gyu Park. Predicting future frames using retrospective cycle gan. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1811–1820, 2019. 1
- [20] Jiahui Lei, Yijia Weng, Adam W Harley, Leonidas Guibas, and Kostas Daniilidis. Mosca: Dynamic gaussian fusion from casual videos via 4d motion scaffolds. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 6165–6177, 2025. 2
- [21] Tianye Li, Mira Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, Steven Lovegrove, Michael Goesele, Richard Newcombe, et al. Neural 3d video synthesis from multi-view video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5521–5531, 2022. 2
- [22] Wenqian Liu, Abhishek Sharma, Octavia Camps, and Mario Sznajder. Dyan: A dynamical atoms-based network for video prediction. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 170–185, 2018. 2
- [23] Chaochao Lu, Michael Hirsch, and Bernhard Scholkopf. Flexible spatio-temporal networks for video prediction. In

- Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6523–6531, 2017. 1
- [24] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis. In *2024 International Conference on 3D Vision (3DV)*, pages 800–809. IEEE, 2024. 2
- [25] Zhanpeng Luo, Haoxi Ran, and Li Lu. Instant4d: 4d gaussian splatting in minutes, 2025. 2
- [26] Weijie Lyu, Xueting Li, Abhijit Kundu, Yi-Hsuan Tsai, and Ming-Hsuan Yang. Gaga: Group any gaussians via 3d-aware memory bank. *arXiv preprint arXiv:2404.07977*, 2024. 2, 3, 4, 6, 1, 5
- [27] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 1, 2
- [28] Ruibo Ming, Zhewei Huang, Zhuoxuan Ju, Jianming Hu, Lihui Peng, and Shuchang Zhou. A survey on future frame synthesis: Bridging deterministic and generative approaches. *arXiv preprint arXiv:2401.14718*, 2024. 1
- [29] Keunhong Park, Utkarsh Sinha, Jonathan T Barron, Sofien Bouaziz, Dan B Goldman, Steven M Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable neural radiance fields. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5865–5874, 2021. 2
- [30] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T Barron, Sofien Bouaziz, Dan B Goldman, Ricardo Martin-Brualla, and Steven M Seitz. Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields. *arXiv preprint arXiv:2106.13228*, 2021.
- [31] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-nerf: Neural radiance fields for dynamic scenes. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10318–10327, 2021. 2, 7, 5
- [32] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 2, 1
- [33] Tianhe Ren, Qing Jiang, Shilong Liu, Zhaoyang Zeng, Wenlong Liu, Han Gao, Hongjie Huang, Zhengyu Ma, Xiaohe Jiang, Yihao Chen, Yuda Xiong, Hao Zhang, Feng Li, Peijun Tang, Kent Yu, and Lei Zhang. Grounding dino 1.5: Advance the “edge” of open-set object detection, 2024. 3, 1
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 5
- [35] Angel Villar-Corrales, Ismail Wahdan, and Sven Behnke. Object-centric video prediction via decoupling of object dynamics and interactions. In *2023 IEEE International Conference on Image Processing (ICIP)*, pages 570–574. IEEE, 2023. 2
- [36] Daniel Wang, Patrick Rim, Tian Tian, Dong Lao, Alex Wong, and Ganesh Sundaramoorthi. Ode-gs: Latent odes for dynamic scene extrapolation with 3d gaussian splatting. *arXiv preprint arXiv:2506.05480*, 2025. 2, 3, 5, 6
- [37] Qianqian Wang, Vickie Ye, Hang Gao, Weijia Zeng, Jake Austin, Zhengqi Li, and Angjoo Kanazawa. Shape of motion: 4d reconstruction from a single video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9660–9672, 2025. 2, 3, 5, 6, 7, 4
- [38] Yunbo Wang, Lu Jiang, Ming-Hsuan Yang, Li-Jia Li, Ming-sheng Long, and Li Fei-Fei. Eidetic 3d LSTM: A model for video prediction and beyond. In *International Conference on Learning Representations*, 2019. 2
- [39] Guanjin Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4d gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 20310–20320, 2024. 2, 6
- [40] Jialong Wu, Shaofeng Yin, Ningya Feng, Xu He, Dong Li, Jianye Hao, and Mingsheng Long. ivideopt: Interactive videopts are scalable world models. *Advances in Neural Information Processing Systems*, 37:68082–68119, 2024. 1
- [41] Yue Wu, Rongrong Gao, Jaesik Park, and Qifeng Chen. Future video synthesis with object motion prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5539–5548, 2020. 1
- [42] Yue Wu, Qiang Wen, and Qifeng Chen. Optimizing video prediction via video frame interpolation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17814–17823, 2022. 1
- [43] Wilson Yan, Yunzhi Zhang, Pieter Abbeel, and Aravind Srinivas. Videopt: Video generation using vq-vae and transformers. *arXiv preprint arXiv:2104.10157*, 2021. 2
- [44] Jinyu Yang, Mingqi Gao, Zhe Li, Shang Gao, Fangjing Wang, and Feng Zheng. Track anything: Segment anything meets videos. *arXiv preprint arXiv:2304.11968*, 2023. 2
- [45] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10371–10381, 2024. 2, 6
- [46] Zeyu Yang, Hongye Yang, Zijie Pan, and Li Zhang. Real-time photorealistic dynamic scene representation and rendering with 4d gaussian splatting. *arXiv preprint arXiv:2310.10642*, 2023. 2
- [47] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3d gaussians for high-fidelity monocular dynamic scene reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 20331–20341, 2024. 2
- [48] Mingqiao Ye, Martin Danelljan, Fisher Yu, and Lei Ke. Gaussian grouping: Segment and edit anything in 3d scenes. In *European conference on computer vision*, pages 162–179. Springer, 2024. 2, 3
- [49] Xi Ye and Guillaume-Alexandre Bilodeau. Vpnr: Efficient transformers for video prediction. In *2022 26th International conference on pattern recognition (ICPR)*, pages 3492–3499. IEEE, 2022. 2

- [50] Boming Zhao, Yuan Li, Ziyu Sun, Lin Zeng, Yujun Shen, Rui Ma, Yinda Zhang, Hujun Bao, and Zhaopeng Cui. Gaussianprediction: Dynamic 3d gaussian prediction for motion extrapolation and free view synthesis. In *ACM SIGGRAPH 2024 Conference Papers*, pages 1–12, 2024. [1](#), [2](#), [3](#), [5](#), [6](#), [7](#)
- [51] Shijie Zhou, Haoran Chang, Sicheng Jiang, Zhiwen Fan, Zehao Zhu, Dejia Xu, Pradyumna Chari, Suyu You, Zhangyang Wang, and Achuta Kadambi. Feature 3dgs: Supercharging 3d gaussian splatting to enable distilled feature fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21676–21685, 2024. [2](#)
- [52] M. Zwicker, H. Pfister, J. van Baar, and M. Gross. Ewa splatting. *IEEE Transactions on Visualization and Computer Graphics*, 8(3):223–238, 2002. [3](#)

Space-Time Forecasting of Dynamic Scenes with Motion-aware Gaussian Grouping

Supplementary Material

Contents

A. Details of Gaussian Grouping

- A.1. Preliminary: Static Gaussian Grouping
- A.2. Naive Extension of Static Gaussian Grouping
- A.3. Motion-aware Gaussian Grouping

B. Implementation Details

- B.1. 4DGS Initialization via SoM
- B.2. Motion-aware Gaussian Grouping
- B.3. Group-wise Optimization
- B.4. Group-wise Motion Forecasting

C. Additional Evaluation Results

- C.1. Future View Synthesis
- C.2. Novel View Synthesis

D. Component Analysis of MoGaF

- D.1. Analysis for Group-wise Design
- D.2. Effect of Motion-aware Gaussian Grouping

E. Limitations and Future Work

A. Details of Gaussian Grouping

In this section, we provide the detailed algorithms used in our Gaussian grouping pipeline, complementing the description in Section 4.1 of the main paper. We first summarize the static Gaussian grouping mechanism used in Gaga [26] (Section A.1), then describe its naive 4D extension (Section A.2), and finally introduce our motion-aware grouping algorithm that incorporates spatiotemporal cues (Section A.3).

A.1. Preliminary: Static Gaussian Grouping

Gaga [26] formulates static 3D Gaussian grouping as the problem of assigning each 3D Gaussian to group label using multi-view 2D instance masks. Given a set of static 3D Gaussians $\mathcal{G}$ reconstructed from a scene and multi-view images with segmentation masks, each 2D mask M is first associated with Gaussians whose projections fall inside the mask region:

$$\mathcal{G}(M_i^{(k)}) = \{g \in \mathcal{G} \mid \text{Proj}(g) \in M_i^{(k)}\}, \quad (13)$$

where $M_i^{(k)}$ is k -th mask detected in i -th image.

3D-aware memory bank. To enforce cross-view consistency, Gaga employs memory banks storing Gaussian groups $\{\mathcal{G}^{(k)}\}$, where each group represents an object-level region in 3D space. At initialization, the masks from the first view are each inserted into the memory bank as separate groups by setting $\mathcal{G}^{(k)} \leftarrow \mathcal{G}(M_0^{(k)})$ for every mask $M_0^{(k)}$ in the initial view. For subsequent views, new masks are either merged into one of these

existing groups or used to create additional groups, depending on their overlap with the current memory contents.

Group ID assignment. For a new mask $m = M_i^{(j)}$ extracted from i -th input image, the similarity between its associated Gaussians $\mathcal{G}(m)$ and an existing group $\mathcal{G}^{(k)}$ is computed using the shared-Gaussian overlap:

$$\text{Overlap}(m, k) = \frac{\#(\mathcal{G}^{(k)} \cap \mathcal{G}(m))}{\#(\mathcal{G}(m))}. \quad (14)$$

This formulation depends only on the newly observed set $\mathcal{G}(m)$, so the threshold is independent of how many Gaussians have already been accumulated in $\mathcal{G}^{(k)}$, avoiding the need to continually retune it as the memory bank grows.

Let $k^* = \arg \max_k \text{Overlap}(m, k)$ be the group with the highest overlap score. If $\text{Overlap}(m, k^*)$ exceeds a fixed threshold θ_{ov} , Gaussians in $\mathcal{G}(m)$ are merged into group $\mathcal{G}^{(k^*)}$:

$$\mathcal{G}^{(k^*)} \leftarrow \mathcal{G}^{(k^*)} \cup \mathcal{G}(m). \quad (15)$$

Otherwise, a new group is created in the memory bank with $\mathcal{G}(m)$ as its initial Gaussian set. In practice, each Gaussian is assigned to at most one group by tracking canonical indices, so once a Gaussian has been inserted into a group, it is not duplicated elsewhere.

Static setting. Because Gaga operates on static scenes, each Gaussian maintains a fixed 3D position, making 2D projection overlap a reliable cue for multi-view object grouping. The method produces coherent object-part segmentation and provides the foundation for extending grouping to dynamic scenes in our work.

A.2. Naive Extension of Static Gaussian Grouping

4D-aware Memory Bank. Given a 2D mask $M_t^{(k)}$ at timestep t , we extract deformed Gaussians contributing to the mask:

$$\mathcal{G}_t^{(k)} = \{g \in \mathcal{G} \mid \text{Proj}(g_t) \in M_t^{(k)}\}, \quad (16)$$

where g_t denotes the deformed state of g . We extend the static memory bank to maintain motion groups over time. At the first frame, the k -th memory bank is initialized as

$$\mathcal{M}^{(k)} = \left(\mathcal{G}_t^{(k)}, \tau_{\text{mask}}^{(k)} \right), \quad (17)$$

where $\tau_{\text{mask}}^{(k)}$ denotes the rigidity type of the segmented region.

Group ID Assignment. Different from Gaga [26], we have access to consistent mask identities provided by video segmentation models [32, 33], which serve as a temporal cue for associating $\mathcal{G}_t^{(k)}$ with the same object across frames. Therefore, we do not apply the max-IoU-based merging strategy used for label-agnostic

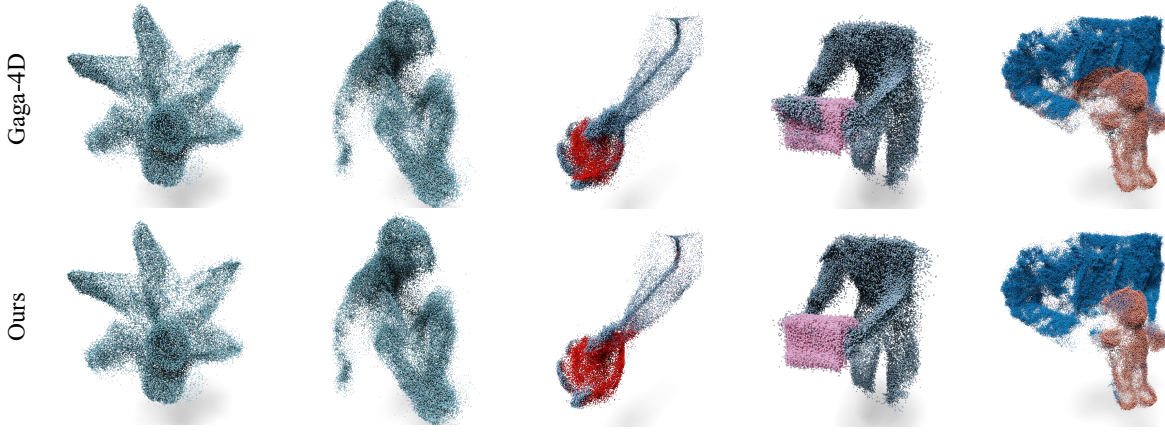


Figure 8. **Gaussian grouping results.** We present Gaussian grouping results of our method and Gaga-4D on iPhone dataset [10], trained on the full sequence.

Algorithm 2: *Naive 4D Gaussian Grouping*

Input: Frames $\{I_t\}$, segmentation model $\mathcal{S}$, Gaussians $\mathcal{G}$, deformed states $\{g_t\}$, overlap threshold θ_{ov}

Output: Groups $\{\mathcal{M}^{(k)} = (G^{(k)}, \tau^{(k)})\}$

Init (first keyframe): Obtain

$\{M_{t_0}^{(k)}, \tau_{mask}^{(k)}\} = \mathcal{S}(I_{t_0})$; **for** $k = 1..K$ **do**
 $G^{(k)} \leftarrow \{g \in \mathcal{G} \mid \text{Proj}(g_{t_0}) \in M_{t_0}^{(k)}\};$
 $\tau^{(k)} \leftarrow \tau_{mask}^{(k)};$

Define canonical ID $\text{id}(g)$ for all g .

Iterative grouping across frames: **for** $t = 1..T$ **do**

Obtain $\{M_t^{(k)}, \tau_{mask}^{(k)}\};$
for $k = 1..K$ **do**
 $G_t^{(k)} \leftarrow \{g \in \mathcal{G} \mid \text{Proj}(g_t) \in M_t^{(k)}\};$
 Compute $\text{Overlap}(t, k)$ for all k using canonical IDs;
if $\text{Overlap}(t, k) \geq \theta_{ov}$ **then**
 $G^{(k)} \leftarrow G^{(k)} \cup G_t^{(k)};$

matching in static Gaussian segmentation. However, dynamic deformation and occlusions may produce unreliable matches, so we verify consistency using the shared-Gaussian overlap:

$$\text{Overlap}(t, k) = \frac{\#(G^{(k)} \cap G_t^{(k)})}{\#(G_t^{(k)})}. \quad (18)$$

The intersection is defined through canonical identity:

$$G^{(k)} \cap G_t^{(k)} = \{g \in G_t^{(k)} \mid \exists g' \in G^{(k)}, \text{id}(g) = \text{id}(g')\}. \quad (19)$$

If $\text{Overlap}(t, k) \geq \theta_{ov}$, we merge:

$$G^{(k)} \leftarrow G^{(k)} \cup G_t^{(k)}. \quad (20)$$

Limitation of naive 4D extension. Because this naive extension relies solely on instantaneous 2D projection overlap, Gaussian deformation from motion optimization (e.g., occlusions, drift, partial visibility) can cause inconsistent overlap patterns. As a result, Gaussians belonging to the same object may fail to merge, fragmenting a single object into multiple groups, while unrelated objects may be mistakenly merged due to projected overlap. This makes the naive extension fragile under ambiguous projections (Figure 3a of the main paper), highlighting the need for motion-aware grouping that leverages temporal and motion cues beyond simple projection overlap.

A.3. Motion-aware Gaussian Grouping

To address the limitations of the naive 4D extension, we propose a spatiotemporal Gaussian grouping algorithm tailored for pre-optimized 4DGS representations. Our method follows an alternating strategy composed of (i) keyframe-based seeding and (ii) feature-space region growing. We first sample T_{key} keyframes from the input video, forming the keyframe set $\{I_t\}_{t=1}^{T_{\text{key}}}$, and extract Gaussians that contribute to the mask rendering at these frames.

Iterative Region Growing Strategy. We initialize motion groups $\mathcal{M}^{(k)} = \{(G^{(k)}, \tau^{(k)})\}$ by extracting the front Gaussians at a set of keyframes following Equation (16). After initialization, we propagate group assignments to neighboring Gaussians via a spatiotemporal region-growing process.

Each Gaussian $g \in \mathcal{G}$ is represented by a spatiotemporal feature vector

$$\mathbf{f}_g = [\boldsymbol{\mu}_{c,g}, \mathbf{w}'_g], \quad (21)$$

where $\boldsymbol{\mu}_{c,g}$ is the canonical-space mean of g , and $\mathbf{w}'_g$ denotes its PCA-reduced motion coefficient.

Given these features, each group $G^{(k)}$ is expanded by aggregating Gaussians in its feature-space neighborhood. For every $g \in G^{(k)}$, we consider its nearest neighbors and include any candidate g' if

$$\|\mathbf{f}_g - \mathbf{f}_{g'}\| < \epsilon_r, \quad g' \notin G^{(k)}. \quad (22)$$

Algorithm 3: Complete Procedure of Motion-aware Gaussian Grouping

Input: Video frames $\{I_t\}_{t=1}^T$, segmentation model $\mathcal{S}$, dynamic Gaussian set $\mathcal{G}$ with deformed states $\{g_t\}_{t=1}^T$

Output: Motion groups $\{\mathcal{M}^{(k)} = (G^{(k)}, \tau^{(k)})\}_{k=1}^K$

Stage 1: Init at the First Keyframe

Obtain $\{M_t^{(k)}, \tau_{\text{mask}}^{(k)}\} \leftarrow \mathcal{S}(\{I_t\})$

Select first keyframe t_k

foreach $k = 1, \dots, K$ **do**

$G^{(k)} \leftarrow \{g \in \mathcal{G} \mid \text{Proj}(g_{t_k}) \in M_{t_k}^{(k)}\}$
 $\tau^{(k)} \leftarrow \tau_{\text{mask}}^{(k)}$

Define features for all g : $\mathbf{f}_g = [\mu_{c,g}, \mathbf{w}'_g]$

Stage 2: Iterative Region Growing

foreach keyframe t in stride order **do**

// (A) Feature-space region growing

repeat

$U \leftarrow \mathcal{G} \setminus \bigcup_k G^{(k)}$

foreach $k = 1, \dots, K$ **do**

$\epsilon_k \leftarrow \alpha \cdot \text{mean}_{g \in G^{(k)}} [\text{KNN}_K(\mathbf{f}_g)]$
 $N_k \leftarrow \{g' \in U \mid \min_{g \in G^{(k)}} \|\mathbf{f}_g - \mathbf{f}_{g'}\| < \epsilon_k\}$
 $G^{(k)} \leftarrow G^{(k)} \cup N_k$
 $U \leftarrow U \setminus N_k$

until no group changes;

// (B) Seeding & merge at keyframe t

foreach $k = 1, \dots, K$ **do**

$G_t^{(k)} \leftarrow \{g \mid \text{Proj}(g_t) \in M_t^{(k)}\}$
 $G^{(k)} \leftarrow G^{(k)} \cup G_t^{(k)}$

Stage 3: Reassign Unassigned Gaussians (KNN Voting)

$G^{\text{labeled}} \leftarrow \bigcup_k G^{(k)}, U \leftarrow \mathcal{G} \setminus G^{\text{labeled}}$

foreach $g_u \in U$ **do**

$\mathcal{N}(g_u) \leftarrow \text{KNN}_K(\mathbf{f}_{g_u}, G^{\text{labeled}})$

foreach $k = 1, \dots, K$ **do**

$v_k(g_u) \leftarrow \sum_{g' \in \mathcal{N}(g_u)} \mathbb{I}[g' \in G^{(k)}]$

$k^*(g_u) = \arg \max_k v_k(g_u)$

$G^{(k^*(g_u))} \leftarrow G^{(k^*(g_u))} \cup \{g_u\}$

This update is repeated until convergence. We then perform a new seeding step at the next keyframe and apply region growing again. By alternating between these two stages, the grouping progressively stabilizes and captures coherent spatiotemporal structure across the sequence.

Handling Unassigned Gaussians. After the alternating seeding–region-growing stages, a small number of Gaussians may remain unassigned because they were not included in any feature-space neighborhood expansion. To relabel these remaining Gaussians, we perform a feature-space voting procedure using only group-labeled Gaussians.

For each unassigned Gaussian g_u , we retrieve its K nearest labeled neighbors in feature space:

$$\mathcal{N}(g_u) = \text{KNN}_K(\mathbf{f}_{g_u}, \mathcal{G}^{\text{labeled}}), \quad (23)$$

where $\mathbf{f}_{g_u}$ is the spatiotemporal feature of g_u and $\mathcal{G}^{\text{labeled}} = \bigcup_k G^{(k)}$ denotes the set of Gaussians with assigned group labels. Among these labeled neighbors, we count how many belong to each motion group. Let $\mathbb{I}[g' \in G^{(k)}]$ denote an indicator function that evaluates to 1 if g' belongs to group k , and 0 otherwise. The voting score for group k is computed as

$$v_k(g_u) = \sum_{g' \in \mathcal{N}(g_u)} \mathbb{I}[g' \in G^{(k)}]. \quad (24)$$

The final group assignment for g_u is then obtained by majority voting:

$$k^*(g_u) = \arg \max_k v_k(g_u). \quad (25)$$

We then update the motion group as

$$G^{(k^*)} \leftarrow G^{(k^*)} \cup \{g_u\}. \quad (26)$$

This voting-based reassignment ensures that isolated or boundary Gaussians are consistently grouped according to their local spatiotemporal context.

As shown in Figure 8, our grouping algorithm works robustly across multiple objects. It produces clear and stable group assignments even in scenes where distinct objects exhibit significant spatial overlap in the input video.

B. Implementation Details

B.1. 4DGS Initialization via SoM

Our method builds on pre-optimized 4DGS representation following Shape-of-Motion (SoM) [37], trained for 300 epochs. We use the same optimization hyperparameters for all benchmark datasets, following the official SoM implementation. For the future view synthesis experiments, we use only the first 60% or 80% of the input sequence to evaluate extrapolation performance.

B.2. Motion-aware Gaussian Grouping

Iterative Region Growing. We implement our grouping strategy by modifying the official Gaga [26] implementation. During initialization and keyframe-based seeding, we extract the closest 40% front Gaussians with respect to the camera view. For the region-growing stage, each Gaussian is represented as a spatiotemporal feature vector $[\mu_{c,g}, \mathbf{w}'_g]$, composed of its canonical-space mean and a PCA-reduced motion coefficient. We apply PCA to the motion coefficients of all dynamic Gaussians in 4DGS representation, scale the resulting PCA components by 0.3, and concatenate them with the Gaussian means. Before performing region growing for each group, we determine the growing radius ϵ_r by computing the average neighbor distance among Gaussians belonging to the same group, ensuring that the expansion scale reflects the underlying local density.

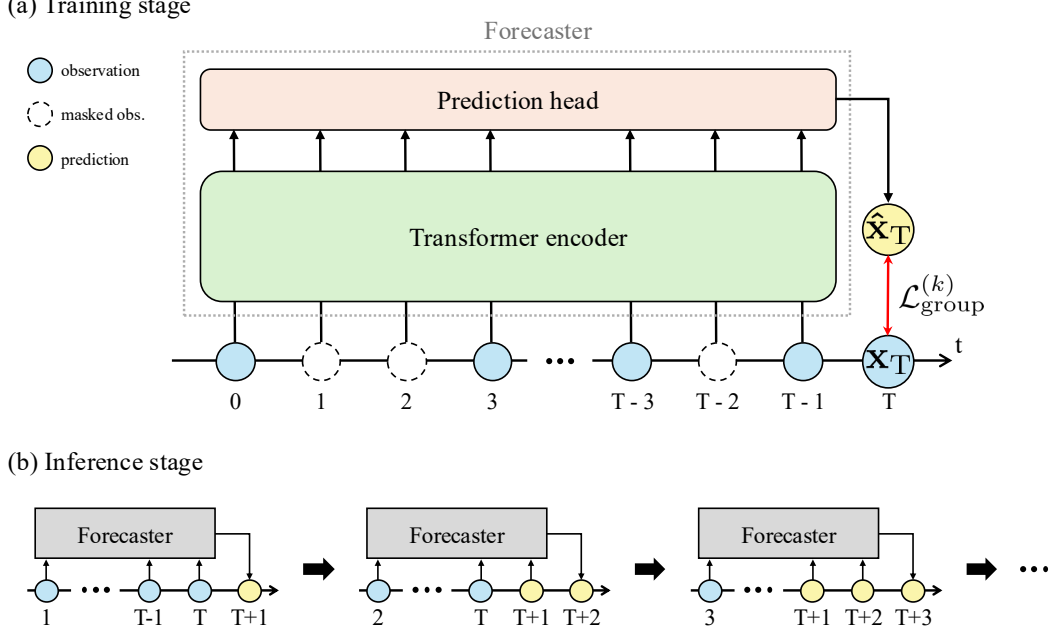


Figure 9. **Overview of the forecaster.** (a) *Training stage*: The forecaster is trained for each Gaussian group $G^{(k)}$ by minimizing the loss $\mathcal{L}_{\text{group}}^{(k)}$ between the predicted $\hat{\mathbf{x}}_T$ and the observed $\mathbf{x}_T$ at time T . We randomly apply contiguous masking to the inputs and gradually reduce the masking ratio later in training. (b) *Inference stage*: Future motion is generated via autoregressive rollout for each $G^{(k)}$, where the input sequence is iteratively updated with each newly predicted frame. Note that the input length is always $T-1$, the same as in training.

Reassignment Strategy. In addition, during the Gaussian re-labeling step for unassigned Gaussians, each Gaussian updates its group label by voting among the 8 nearest group-assigned neighbors. To further enforce reliable and spatially consistent grouping, we then reassign only the closest 50% of unassigned Gaussians, determined by their average distance to the neighboring group-assigned Gaussians.

B.3. Group-wise Optimization

Our group-wise optimization consists of two main stages: (i) rigid motion initialization, and (ii) group-wise refinement. In the first stage, we initialize rigid group motion parameters to enable our motion-aware objective. In the second stage, using the initialized motion information, we refine pre-optimized 4DGS representation to enforce stronger spatiotemporal consistency.

Rigid Motion Initialization. Before refining 4DGS representation, we first estimate the rigid body transformation parameters for rigid groups (i.e., groups with $\tau^{(k)} = 1$) to implement our rigid motion anchoring loss. Specifically, for each rigid motion group, we compute a sequence of rigid transformations $\{\Phi_t^{(k)} = [\mathbf{R}_{c \rightarrow t}^{(k)} \mid \mathbf{t}_{c \rightarrow t}^{(k)}]\}_{t=1}^T$ by solving a Procrustes alignment problem. The alignment at timestep t is given by

$$\Phi_t^{(k)} = \arg \min_{\mathbf{R}_{c \rightarrow t}^{(k)}, \mathbf{t}_{c \rightarrow t}^{(k)}} \sum_{g \in G^{(k)}} \left\| \mathbf{R}_{c \rightarrow t}^{(k)} \boldsymbol{\mu}_{c,g} + \mathbf{t}_{c \rightarrow t}^{(k)} - \boldsymbol{\mu}_{t,g} \right\|_2^2, \quad (27)$$

where $\mathbf{R}_{c \rightarrow t}^{(k)} \in \text{SO}(3)$ and $\mathbf{t}_{c \rightarrow t}^{(k)} \in \mathbb{R}^3$. Here, $\boldsymbol{\mu}_{c,g}$ and $\boldsymbol{\mu}_{t,g}$ denote the canonical and deformed Gaussian centers.

Refinement of 4DGS Representation. Similar to the pre-optimization stage (Section B.1), our refinement is also implemented on top of SoM. We incorporate our group-wise motion optimization objective $\mathcal{L}_{\text{motion}}$ into the original training loss. For optimization stability, we apply scaling weights and reformulate the objective as:

$$\mathcal{L}_{\text{motion}} = \sum_{k=1}^K \left[\tau^{(k)} \cdot \lambda_{\text{rigid}} \mathcal{L}_{\text{rigid}}^{(k)} + (1 - \tau^{(k)}) \cdot \lambda_{\text{nr}} \mathcal{L}_{\text{nr}}^{(k)} \right], \quad (28)$$

where we set $\lambda_{\text{rigid}} = 0.1$ and $\lambda_{\text{nr}} = 0.02$ in all experiments.

We apply our constraint optimization objective together with the original reconstruction loss of SoM [37], defined as

$$\mathcal{L}_{\text{recon}} = \|\hat{\mathbf{I}} - \mathbf{I}\|_1 + \lambda_{\text{depth}} \|\hat{\mathbf{D}} - \mathbf{D}\|_1 + \lambda_{\text{mask}} \|\hat{\mathbf{M}} - \mathbf{M}\|_1, \quad (29)$$

where $\hat{\mathbf{I}}, \hat{\mathbf{D}}, \hat{\mathbf{M}}$ denote the rendered RGB, depth, and mask predictions from current 4DGS representation. We follow the loss-weight configuration provided in the official SoM implementation. Our final optimization objective is then formulated as

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{motion}}. \quad (30)$$

This combined objective ensures that the 4DGS representation remains photometrically faithful while also enforcing coherent group-wise rigid and non-rigid motion behavior.

B.4. Group-wise Motion Forecasting

Architecture Details. We illustrate the architecture of our forecaster in Figure 9. The model uses a single-layer Transformer encoder with a 32-dimensional hidden state, eight attention heads,

Table 4. **Forecasting results on the iPhone dataset.** We forecast frames beyond the observed time window and evaluate the predicted frames rendered from held-out test viewpoints. The leftmost column (*Obs. ratio*) indicates the fraction of input training frames used by each model.

<i>Obs. ratio</i>	Scene	GSPred			GSPred-SoM			MoGaF (Ours)		
		mPSNR↑	mSSIM↑	mLPIPS↓	mPSNR↑	mSSIM↑	mLPIPS↓	mPSNR↑	mSSIM↑	mLPIPS↓
70%	apple	13.21	0.5562	0.6942	16.58	0.7339	0.5007	16.45	0.7220	0.5329
	block	13.11	0.5333	0.5240	13.47	0.6192	0.4900	14.10	0.6110	0.5030
	paper-windmill	14.67	0.2168	0.4410	18.73	0.5221	0.2191	18.98	0.5197	0.2108
	spin	15.67	0.4738	0.3975	16.00	0.6797	0.3049	15.87	0.6618	0.3438
	teddy	11.46	0.5886	0.5708	10.84	0.5666	0.6555	11.87	0.5562	0.5578
	Average	13.62	0.4737	0.5255	15.12	0.6243	0.4341	15.45	0.6141	0.4296
90%	apple	15.21	0.5550	0.4574	16.88	0.8541	0.4453	16.98	0.8526	0.4530
	block	14.57	0.5445	0.4909	12.49	0.5896	0.5363	14.19	0.6018	0.4912
	paper-windmill	14.78	0.2201	0.4280	19.00	0.5559	0.2111	19.22	0.5538	0.2036
	spin	14.75	0.4631	0.4232	16.14	0.6931	0.3146	16.99	0.6975	0.2933
	teddy	11.37	0.5816	0.5929	11.99	0.5411	0.5894	12.39	0.5433	0.5436
	Average	14.13	0.4728	0.4785	15.30	0.6467	0.4193	15.95	0.6498	0.3970

and a 64-dimensional feedforward layer, followed by a flatten-head decoder. We apply a dropout rate of 0.2 and use instance normalization for each sequence. For both training and inference, we represent each Gaussian motion $\mathbf{x}_t$ at time t as

$$\mathbf{x}_t = [\boldsymbol{\mu}_t, \mathbf{q}_t] \in \mathbb{R}^7, \quad (31)$$

where $\boldsymbol{\mu}_t \in \mathbb{R}^3$ denotes the 3D translation vector and $\mathbf{q}_t \in \mathbb{R}^4$ denotes the unit quaternion that parameterizes the rotation.

Training Procedure and Inference Scheme. During training, we employ variable observation masking, where a contiguous span covering 40–0% of the sequence is masked, and the masking ratio is gradually annealed over the course of training. To encourage physically plausible and smooth motion, we apply an acceleration regularization term with $\lambda_{\text{acc}} = 1.0$ for all experiments.

We further adopt a group-wise forecasting scheme in which one forecaster is trained per motion group, and each forecaster is responsible for training and predicting only the trajectories associated with its assigned group.

C. Additional Evaluation Results

C.1. Future View Synthesis

Evaluation on the iPhone Dataset. For the iPhone dataset [10], we provide quantitative evaluation results for additional observation ratios (70% and 90%) in Table 4. We further present additional qualitative results for long-term forecasting (Figure 11) and qualitative comparisons across diverse scenes (Figure 12), extending Figure 4.

Evaluation on D-NeRF dataset. We further report evaluation results on D-NeRF dataset [31]. Using the same setup as GSPred [50], we use an 80% observation ratio and a 20% test split. In this setting, Gaussian motions are forecasted, and all trajectories are extrapolated using a weighted sum of the predicted motion. The quantitative results are summarized in Table 5. Additionally, qualitative comparisons against GSPred are presented in Figure 13.

Table 5. **Forecasting results on D-NeRF dataset.** We use 80% of the frames as input and predict the remaining 20%.

Scene	GSPred [50]			Ours		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Trex	21.09	0.9406	0.0461	20.60	0.9405	0.0499
Mutant	28.16	0.9560	0.0256	28.72	0.9552	0.0218
Jumpingjacks	20.51	0.9184	0.0760	20.12	0.9220	0.0734
Standup	25.96	0.9403	0.0481	30.38	0.9544	0.0335
Bouncingballs	26.63	0.9714	0.0361	26.82	0.9728	0.0367
Hook	23.42	0.9089	0.0573	24.13	0.9081	0.0562
Hellwarrior	30.75	0.9281	0.0729	31.29	0.9323	0.0680
Lego	11.99	0.7562	0.2338	24.88	0.9005	0.0532
Average	23.56	0.9150	0.0745	25.87	0.9357	0.0491

C.2. Novel View Synthesis

To analyze the effect of our group-wise optimization on scene interpolation, we evaluate novel view synthesis (NVS) results and point-tracking performance on iPhone dataset. The experiment is conducted using the full training set (100% observation ratio) so that the evaluation reflects pure interpolation performance.

As reported in Table 6, our constrained optimization yields slight improvements in both point-tracking accuracy and photometric metrics. Qualitatively, Figure 10 shows that our method enhances the geometric consistency of resulting 4DGS representation. These results confirm that our Gaussian grouping and group-wise constrained optimization help obtain more physically plausible dynamic Gaussian representation.

D. Component Analysis of MoGaF

For a deeper analysis of our pipeline, we conduct a component-wise analysis of MoGaF and evaluate the effectiveness of our motion-aware Gaussian grouping strategy. The component analysis focuses on three key design axes: (1) group-wise optimization, (2) the group-wise forecasting scheme, and (3) the capacity of the motion forecaster. For the grouping strategy, we compare our method with a simple extension of static Gaussian grouping [26]. All experiments are conducted with a 60% observation ratio, forecasting the remaining 40% of each sequence on iPhone dataset [10].

Table 6. **Quantitative evaluation results on scene interpolation.** We compare tracking and photometric performance against SoM [37] on iPhone dataset [10].

Methods	3D tracking			2D tracking			Photometric		
	EPE↓	$\delta_{3D}^{10}\uparrow$	$\delta_{3D}^{05}\uparrow$	AJ↑	$\delta_{avg}\uparrow$	OA↑	MPSNR↑	mSSIM↑	mLPIPS↓
SoM [37]	0.1016	70.62	46.59	37.61	49.17	86.99	16.72	0.6462	0.3914
MoGaF (Ours)	0.0989	70.63	47.64	38.54	49.93	86.85	16.75	0.6462	0.3887

Table 7. **Ablation studies on group-wise design and region growing.** We analyze the contribution of each component in our method across 3D/2D motion tracking and novel view synthesis.

(a) Effect of group-wise optimization, forecasting, and encoder capacity on future Gaussian motion forecasting. The gray row denotes MoGaF (Ours).

Encoder	Group-wise		3D tracking			2D tracking		
	Opt.	Forecast	EPE↓	$\delta_{3D}^{10}\uparrow$	$\delta_{3D}^{05}\uparrow$	AJ↑	$\delta_{avg}\uparrow$	OA↑
5-Layer (HC)	—	—	0.253	37.5	12.7	12.8	7.9	75.9
	✓	—	0.264	40.0	15.8	13.0	7.6	77.8
	—	✓	0.246	42.6	16.7	16.0	8.5	74.8
	✓	✓	0.245	43.5	19.5	16.1	9.4	79.2
1-Layer (LW)	—	—	0.296	35.6	17.1	13.3	8.5	64.1
	✓	—	0.269	36.7	15.8	13.4	7.6	70.5
	—	✓	0.237	42.9	16.9	15.4	8.6	72.2
	✓	✓	0.236	44.8	22.5	17.5	8.8	80.1

(b) Forecasting results comparing a naive extension (Gaga-4D) and our motion-aware Gaussian grouping.

Method	mPSNR ↑	mSSIM ↑	mLPIPS ↓	$\delta_{3D}^{10}\uparrow$	$\delta_{3D}^{05}\uparrow$	OA ↑
Gaga-4D	15.31	0.6015	0.4475	43.4	20.9	75.6
Ours	15.51	0.6143	0.4245	44.8	22.5	80.1

D.1. Analysis for Group-wise Design

To isolate the impact of each component, we disable the group-wise optimization and group-wise forecasting modules individually. We further evaluate a higher-capacity motion forecaster—a 5-layer transformer encoder (HC)—against our default 1-layer transformer (LW) encoder. To accurately evaluate forecasting performance, we compare various configurations using point tracking accuracy.

Effect of Group-wise Optimization. Table 7a summarizes the results of our component analysis. For both HC and LW forecasters, enabling group-wise optimization consistently improves 3D tracking accuracy. This suggests that object-level motion constraints help preserve coherent and stable spatiotemporal trajectories within 4DGS representation.

Contribution of Group-wise Forecasting. Group-wise forecasting further improves performance across all metrics. By learning motion patterns within each object independently, the forecaster produces more stable long-term predictions and higher 2D tracking accuracy. When combined with group-wise optimization, it yields the best results for both HC and LW settings, highlighting the complementary roles of the two modules.

Effect of Forecaster Capacity. Interestingly, the HC forecaster does not provide noticeable improvement over the LW ver-

sion. We hypothesize that larger models may be more sensitive to noisy or frame-specific motion variations in 4DGS, tend to introduce instability in long-term predictions. In contrast, the lightweight forecaster appears to leverage the structured, object-level motion units more effectively, suggesting that strong forecasting performance is achievable without large model capacity.

D.2. Effect of Motion-aware Gaussian Grouping

We evaluate the effectiveness of our motion-aware Gaussian grouping strategy for scene forecasting. Specifically, we compare our approach with a baseline that applies iterative region growing based on a naive extension of static Gaussian grouping (Gaga-4D), which is described in Section A.2.

As shown in Table 7b, our method consistently outperforms the baseline across all evaluation metrics, including photometric metrics and point tracking accuracy. These results indicate that our motion-aware grouping enables more accurate separation of Gaussians with coherent motion, leading to improved extrapolation under out-of-distribution (OOD) conditions. Overall, these results demonstrate that our region growing strategy effectively enhances motion awareness in Gaussian grouping under complex dynamic scenes.

E. Limitations and Future Work

Our method exhibits several limitations arising from its dependence on pretrained 4DGS representation. Since the grouping, optimization, and forecasting stages operate on geometry produced by the baseline 4DGS, the overall quality is bounded by the fidelity of this reconstruction. Thus, the method cannot recover unseen geometry, and severe failures in the underlying motion optimization—especially in highly complex scenes—directly propagate to later stages. Incorporating point-cloud completion techniques or generative priors may alleviate this dependency by providing additional geometric constraints.

Another limitation appears when object groups overlap during forecasting. Because Gaussians are not physical primitives, our model cannot reason about collisions or inter-object interactions, offering limited ability to detect or prevent physically implausible overlap between motion groups. Extending the framework with physical constraints or physically grounded Gaussian representations would be a promising direction for future work.

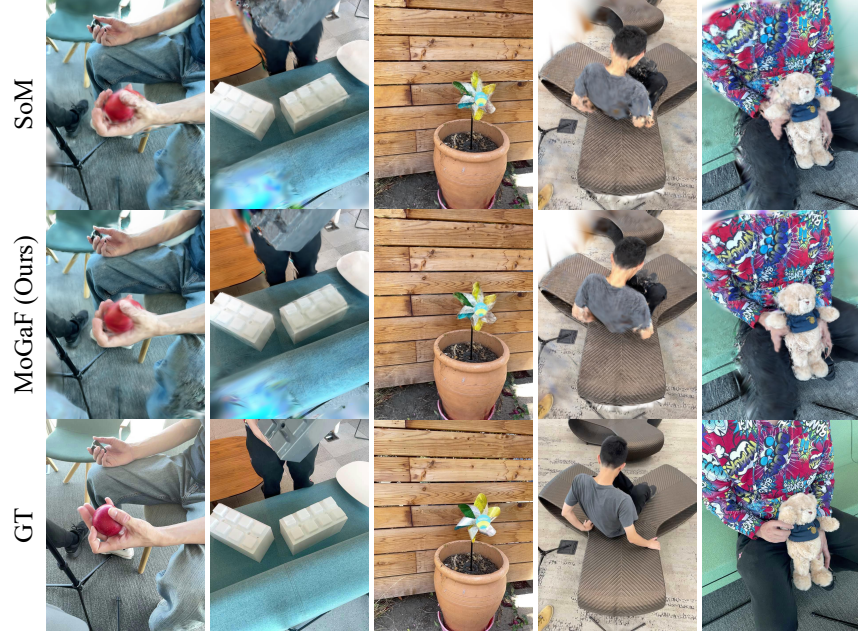


Figure 10. **Qualitative evaluation results on scene interpolation.** We report NVS performance of SoM [37] and MoGaF on iPhone dataset [10], evaluated on test camera viewpoints over the full training sequence.

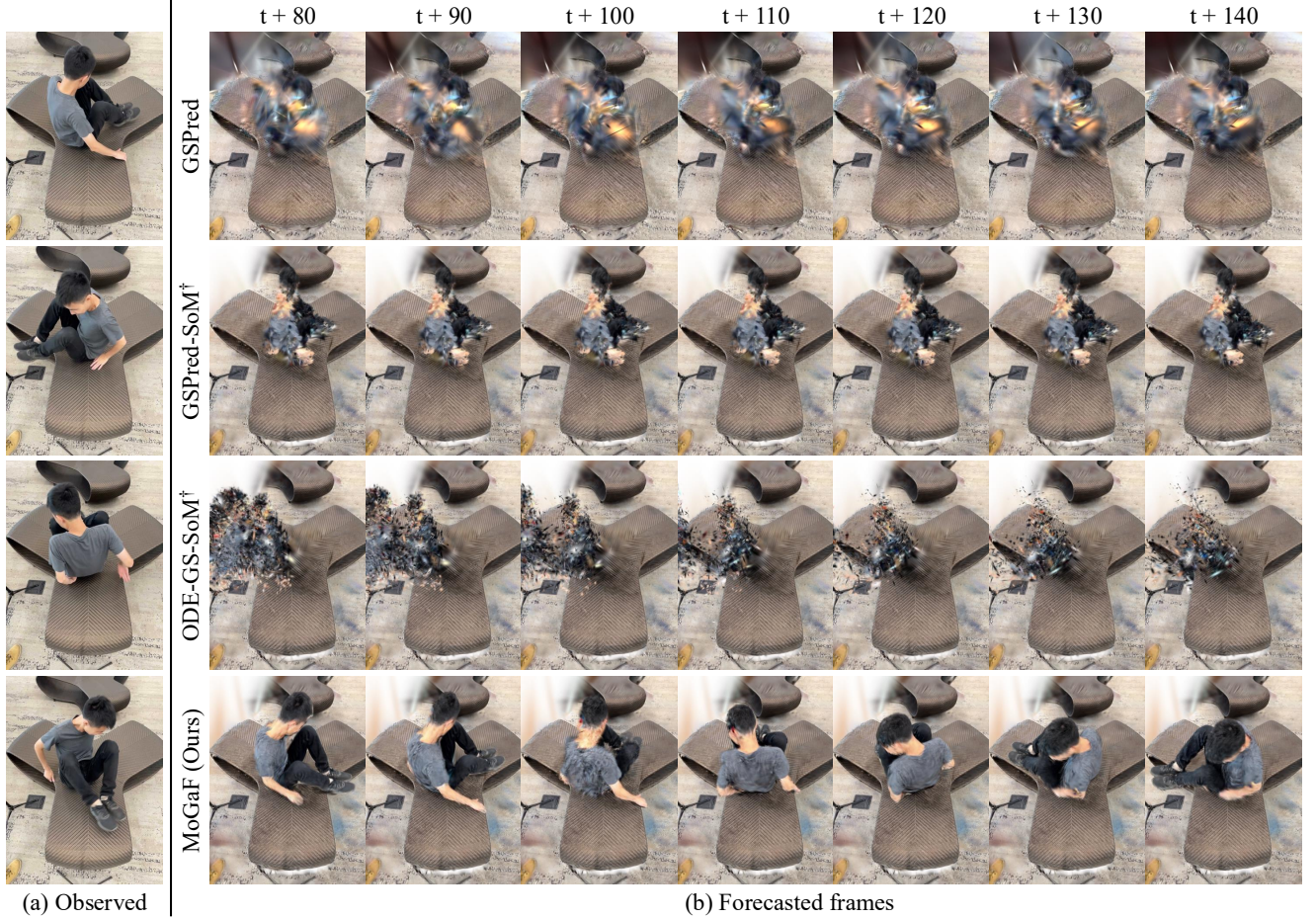


Figure 11. **Long-term forecasting results on iPhone dataset.** (a) shows GT views from test cameras at the observed timesteps, and (b) presents the forecasted renderings for timesteps beyond the observations. Note that t denotes the timestep of the last observed frame.

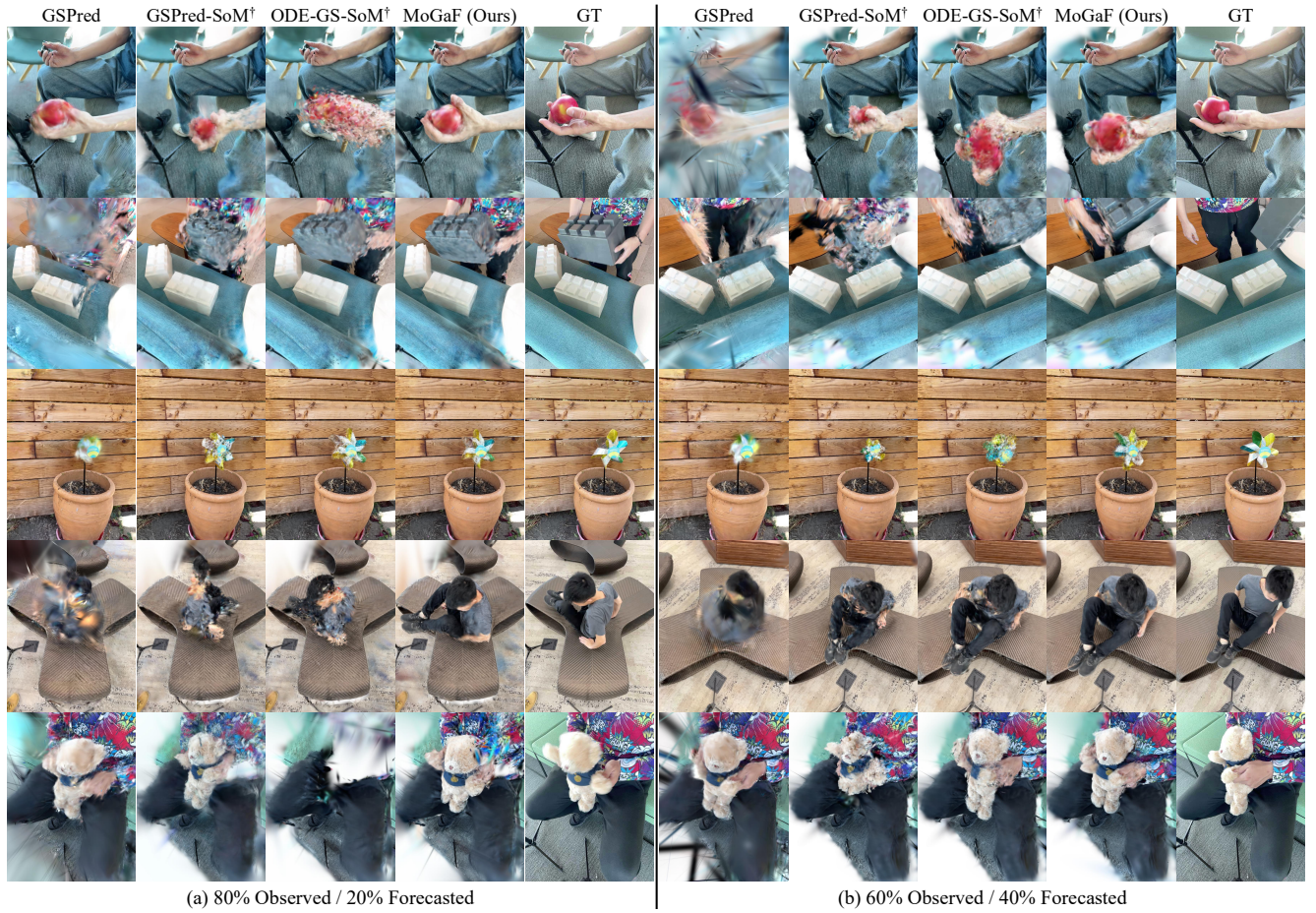


Figure 12. **Qualitative results on iPhone dataset.** We present forecasted frames from test camera views. (a) and (b) correspond to settings where the first 80% and 60% of frames are used for training, and the remaining 20% and 40% are forecasted, respectively.

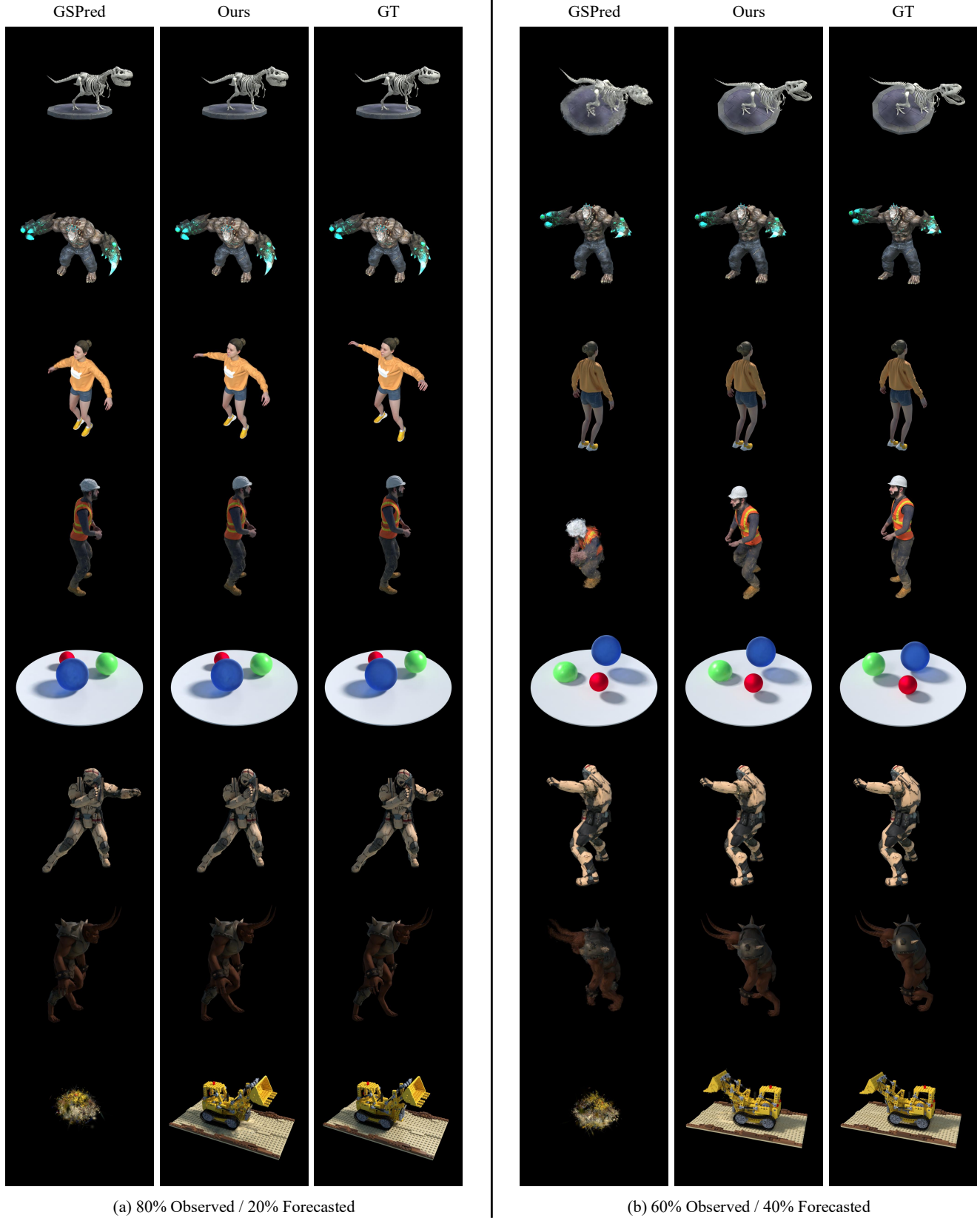


Figure 13. **Qualitative results on D-NeRF dataset.** We present forecasted frames from test camera views. (a) and (b) correspond to settings where the first 80% and 60% of frames are used for training, and the remaining 20% and 40% are forecasted, respectively.